

Contents

KODUS Installation Manual	2
Linux, Nginx, PHP-FPM, and MySQL/MariaDB Deployment	2
1. Purpose	2
2. System Requirements	2
3. Server Prerequisites	2
4. Required Software and Packages	3
5. Folder Structure / Deployment Directory	3
6. Database Setup	4
7. Application File Deployment	5
Step 1. Create the deployment directory	5
Step 2. Copy or clone the application files	5
Step 3. Install PHP dependencies	5
8. Environment and Configuration Settings	5
9. Nginx Server Block Configuration	6
HTTPS note	9
10. PHP-FPM Configuration Notes	9
11. File and Folder Permissions	9
12. Running Database Migrations or SQL Import	10
13. SMTP / Email Configuration	10
14. Security Recommendations	11
15. Testing the Deployment	11
16. Troubleshooting	12
Nginx configuration errors	12
PHP-FPM errors	12
Database connection errors	12
Permission issues	12
URL or routing issues	13
17. Backup and Restore Notes	13
Database backup	13

Application file backup	13
Restore database	13
Restore application files	13
18. Maintenance Notes	13
Deployment Verification Checklist	14

KODUS Installation Manual

Linux, Nginx, PHP-FPM, and MySQL/MariaDB Deployment

1. Purpose

This manual provides the installation and deployment procedure for the KODUS web application on a Linux server running Nginx, PHP-FPM, and MySQL or MariaDB.

This guide is intended for system administrators and deployment personnel responsible for setting up a production or staging environment.

2. System Requirements

- Linux server distribution supported by the company standard build
- Nginx web server
- PHP-FPM for `php version` or later compatible with project dependencies
- MySQL or MariaDB server
- Shell access with `sudo` privileges
- Valid domain or subdomain, for example `your_domain.com`
- SSL/TLS certificate for HTTPS

To be confirmed by project owner:

- Minimum supported PHP version for production
- Official production domain name
- Official deployment path if different from `/var/www/kodus`

3. Server Prerequisites

Before deployment, confirm the following:

- The server has internet access for package installation and dependency download
- The server clock and timezone are configured correctly
- Required firewall ports are open:
 - 80/tcp
 - 443/tcp
- DNS for `your_domain.com` points to the server IP
- A service account or deployment user is available
- MySQL or MariaDB root or administrative credentials are available

4. Required Software and Packages

Example installation on Ubuntu or Debian:

```
sudo apt update
sudo apt install -y nginx mysql-server php-fpm php-mysql php-cli
↪ php-curl php-mbstring php-xml php-zip php-gd php-bcmath php-intl
↪ php-fileinfo unzip curl git composer
```

Recommended PHP extensions based on the current codebase and dependencies:

- mysqli
- mbstring
- openssl
- json
- fileinfo
- curl
- zip
- gd
- xml

Confirm installed versions:

```
nginx -v
php -v
php-fpm -v
mysql --version
composer --version
```

5. Folder Structure / Deployment Directory

Recommended deployment directory:

/var/www/kodus

Suggested structure:

```
/var/www/kodus/
├── admin/
├── crossmatch/
├── deduplication/
├── docs/
├── implementation-status/
├── inbox/
├── notifications/
├── pages/
├── plugins/
├── scripts/
├── sql/
├── storage/
├── vendor/
├── .env
├── composer.json
├── config.php
├── index.php
└── ...
```

Important current deployment note:

- The repository contains routing and links that currently assume the application may be served from /kodus.
- Safest deployment pattern at this time:
 - Web root points to the parent directory containing the kodus folder
 - Application is accessed as `https://your_domain.com/kodus/`
- Root-domain deployment without /kodus should be treated as To be confirmed by project owner.

6. Database Setup

Create the database and application user.

Example for MySQL or MariaDB:

```
CREATE DATABASE database_name CHARACTER SET utf8mb4 COLLATE
↳ utf8mb4_unicode_ci;
```

```
CREATE USER 'database_user'@'localhost' IDENTIFIED BY 'strong_password';
GRANT ALL PRIVILEGES ON database_name.* TO 'database_user'@'localhost';
FLUSH PRIVILEGES;
```

If the database server is remote, adjust the host and access policy as required by company standards.

7. Application File Deployment

Step 1. Create the deployment directory

```
sudo mkdir -p /var/www/kodus
```

Step 2. Copy or clone the application files

Option A: clone from repository

```
cd /var/www  
sudo git clone <repository_url> kodus
```

Option B: copy release package

```
sudo rsync -av /path/to/release/ /var/www/kodus/
```

Step 3. Install PHP dependencies

```
cd /var/www/kodus  
composer install --no-dev --optimize-autoloader
```

If Composer is not available globally:

```
php composer.phar install --no-dev --optimize-autoloader
```

8. Environment and Configuration Settings

Create the environment file from the template:

```
cd /var/www/kodus  
cp .env.example .env
```

Update .env with production values:

```
DB_HOST=127.0.0.1
DB_USERNAME=database_user
DB_PASSWORD=strong_password
DB_NAME=database_name

APP_BASE_PATH=/

SMTP_HOST=smtp.example.com
SMTP_PORT=465
SMTP_USERNAME=your-email@example.com
SMTP_PASSWORD=your-app-password
SMTP_FROM_NAME="KODUS Admin"
SMTP_FROM_ADDRESS=your-email@example.com
```

Optional settings in `.env`:

- Caraga Connect SSO values
- Socket bridge values
- KODA scene values

These should be configured only if used by the deployment environment.

To be confirmed by project owner:

- Whether SSO is required in the Linux deployment
- Whether socket bridge features are required
- Correct `APP_BASE_PATH` value for the final URL structure

9. Nginx Server Block Configuration

Create an Nginx site configuration.

Example:

```
server {
    listen 80;
    server_name your_domain.com;

    root /var/www;
    index index.php index.html;

    client_max_body_size 25m;

    add_header X-Frame-Options "SAMEORIGIN" always;
    add_header X-Content-Type-Options "nosniff" always;
```

```

add_header Referrer-Policy "strict-origin-when-cross-origin" always;

error_page 400 /kodus/400;
error_page 401 /kodus/401;
error_page 403 /kodus/403;
error_page 404 /kodus/404;
error_page 405 /kodus/405;
error_page 408 /kodus/408;
error_page 429 /kodus/429;
error_page 500 /kodus/500;
error_page 502 /kodus/502;
error_page 503 /kodus/503;
error_page 504 /kodus/504;

location ~* ^/(.+)?index\.php$ {
    return 301 /$1;
}

location ~* ^(.+)\.(php|html)$ {
    return 301 /$1;
}

location ~ /\.(!well-known) {
    deny all;
}

location ~* /(?:composer\. (json|lock|phar)|composer-
↳ setup\.php|phpinfo\.php|info\.php|kodus_db\.sql|kodus-
↳ key_pair\. (pem|ppk)|debug_log\.txt|__diag_password_policy\.php|__https_debug\.php)$
↳ {
    deny all;
}

location ~*
↳ \.(?:sql(?:\..*)?)|bak|old|orig|save|log|ini|env|pem|ppk|key)$ {
    deny all;
}

location ~* ^/(?:stor-
↳ age|artifacts|scratch|sql|screenshots|docs|\.git|\.tmp\.driveupload|deduplication/logs)/
↳ {
    deny all;
}

location ~* ^/(?:uploads|inbox/uploads|dist/img)/.*\.(?:php[0-
↳ 9]?|phtml|phar|cgi|pl|exe|sh|bat|cmd)$ {

```

```

        deny all;
    }

    location ~* ^/(?:cross-
↪ match/uploads|deduplication/uploads|inbox/uploads|pages/uploads)/.*\.(?:php[0-
↪ 9]?|phtml|phar|cgi|pl|exe|sh|bat|cmd|js|jsp|asp|aspx)$ {
        deny all;
    }

    location /kodus/ {
        try_files $uri $uri/ $uri.php $uri.html
↪ /kodus/index.php?$query_string;
    }

    location = /kodus/400 { try_files /kodus/400.php =400; }
    location = /kodus/401 { try_files /kodus/401.php =401; }
    location = /kodus/403 { try_files /kodus/403.php =403; }
    location = /kodus/404 { try_files /kodus/404.php =404; }
    location = /kodus/405 { try_files /kodus/405.php =405; }
    location = /kodus/408 { try_files /kodus/408.php =408; }
    location = /kodus/429 { try_files /kodus/429.php =429; }
    location = /kodus/500 { try_files /kodus/500.php =500; }
    location = /kodus/502 { try_files /kodus/502.php =502; }
    location = /kodus/503 { try_files /kodus/503.php =503; }
    location = /kodus/504 { try_files /kodus/504.php =504; }

    location ~ \.php$ {
        include fastcgi_params;
        fastcgi_param SCRIPT_FILENAME
↪ $document_root$fastcgi_script_name;
        fastcgi_pass unix:/run/php/php version-fpm.sock;
    }
}

```

Replace:

- your_domain.com
- php version
- any path values required by the target server

Enable the site and reload Nginx:

```

sudo ln -s /etc/nginx/sites-available/kodus.conf
↪ /etc/nginx/sites-enabled/kodus.conf
sudo nginx -t
sudo systemctl reload nginx

```


HTTPS note

Configure SSL/TLS before production use. Example with Certbot:

```
sudo apt install -y certbot python3-certbot-nginx
sudo certbot --nginx -d your_domain.com
```

10. PHP-FPM Configuration Notes

Check the active PHP-FPM service name:

```
systemctl list-units --type=service | grep fpm
```

Common configuration points:

- Set a suitable `memory_limit`
- Set `upload_max_filesize`
- Set `post_max_size`
- Set `max_execution_time`
- Confirm the correct timezone

Example PHP configuration checks:

```
php -i | grep memory_limit
php -i | grep upload_max_filesize
php -i | grep post_max_size
php -i | grep date.timezone
```

After changes:

```
sudo systemctl restart php version-fpm
sudo systemctl reload nginx
```

11. File and Folder Permissions

Set ownership so Nginx and PHP-FPM can read the application files:

```
sudo chown -R www-data:www-data /var/www/kodus
```

Recommended base permissions:

```
sudo find /var/www/kodus -type d -exec chmod 755 {} \;  
sudo find /var/www/kodus -type f -exec chmod 644 {} \;
```

If writable directories are required for uploads or generated outputs, grant write permission only to those specific paths.

Examples to review and confirm:

- /var/www/kodus/storage
- upload folders under feature directories

To be confirmed by project owner:

- Exact writable folders required in production

12. Running Database Migrations or SQL Import

If a database dump or SQL schema file is provided, import it after creating the database:

```
mysql -u database_user -p database_name < /path/to/kodus_db.sql
```

If the project uses setup helpers that create missing tables during runtime, they may initialize some schema automatically after the first application requests. This should not replace a proper database import for production.

To be confirmed by project owner:

- Official SQL file to import for production
- Whether any manual post-import SQL steps are required

13. SMTP / Email Configuration

KODUS uses SMTP values from `.env`.

Required settings:

- SMTP_HOST
- SMTP_PORT
- SMTP_USERNAME
- SMTP_PASSWORD
- SMTP_FROM_NAME
- SMTP_FROM_ADDRESS

Recommended production practices:

- Use a dedicated application mailbox
- Use an application password or relay credentials
- Restrict outbound SMTP at firewall level as required by company policy

Test mail-related features after deployment:

- Login notification
- Password reset
- Contact or message sending, if enabled

14. Security Recommendations

- Enforce HTTPS for all users
- Keep `.env`, SQL files, logs, and private keys inaccessible from the web
- Do not allow PHP execution in upload directories
- Use least-privilege database credentials
- Disable directory listing
- Keep Linux packages and PHP dependencies updated
- Restrict SSH access by IP and key-based authentication where possible
- Back up the database and uploaded files before updates
- Store secrets only in `.env` or approved secret-management tooling
- Review file upload paths and writable folders before go-live

15. Testing the Deployment

Perform the following checks after deployment:

1. Open `https://your_domain.com/kodus/`
2. Confirm the login page loads
3. Confirm extensionless routes work, for example `/kodus/home`
4. Test user login and logout
5. Test one authenticated page
6. Test database connectivity through the application
7. Test one file upload if the feature is used
8. Test outgoing email
9. Confirm forbidden files such as `.env` are not publicly accessible
10. Review Nginx and PHP-FPM logs for startup or permission errors

16. Troubleshooting

Nginx configuration errors

```
sudo nginx -t
sudo systemctl status nginx
sudo journalctl -u nginx -n 100 --no-pager
```

PHP-FPM errors

```
sudo systemctl status php version-fpm
sudo journalctl -u php version-fpm -n 100 --no-pager
```

Database connection errors

Check:

- .env database values
- database server availability
- database user privileges
- local firewall or bind address settings

Manual connection test:

```
mysql -u database_user -p -h 127.0.0.1 database_name
```

Permission issues

Symptoms:

- Upload failures
- Cannot write files
- HTTP 500 on file operations

Actions:

- Verify owner and group
- Verify directory write permission only where required
- Check Nginx and PHP-FPM logs

URL or routing issues

Check:

- Nginx `try_files` rules
- actual deployment path
- whether the app is being served from `/kodus`
- `APP_BASE_PATH` in `.env`

17. Backup and Restore Notes

Database backup

```
mysqldump -u database_user -p database_name > /backup/kodus_db_$(date  
↪ +%F).sql
```

Application file backup

```
sudo rsync -av /var/www/kodus/ /backup/kodus_files_$(date +%F)/
```

Restore database

```
mysql -u database_user -p database_name <  
↪ /backup/kodus_db_YYYY-MM-DD.sql
```

Restore application files

```
sudo rsync -av /backup/kodus_files_YYYY-MM-DD/ /var/www/kodus/
```

Back up the following before upgrades:

- database
- `.env`
- writable upload directories
- generated output files if operationally required

18. Maintenance Notes

- Apply operating system and package updates during approved maintenance windows
- Restart or reload Nginx and PHP-FPM after configuration changes

- Retain a rollback copy of the previous release
- Re-run `composer install --no-dev --optimize-autoloader` after dependency changes
- Review logs after each deployment
- Validate key business functions after every update
- Keep a current database backup before any schema or application change

Deployment Verification Checklist

- ☐ Linux server is reachable and updated
- ☐ Nginx is installed and running
- ☐ PHP-FPM is installed and running
- ☐ MySQL or MariaDB is installed and accessible
- ☐ Application files are deployed to `/var/www/kodus`
- ☐ Composer dependencies are installed
- ☐ `.env` is present with production values
- ☐ Database `database_name` exists
- ☐ Database import or initialization is complete
- ☐ Nginx server block is enabled and passes `nginx -t`
- ☐ PHP-FPM socket or port in Nginx matches the installed PHP version
- ☐ File ownership and permissions are correct
- ☐ HTTPS is enabled
- ☐ Login page loads successfully
- ☐ Authenticated pages load successfully
- ☐ Email sending works
- ☐ Sensitive files are blocked from public access
- ☐ Logs show no unresolved fatal errors after testing